

语法制导定义 (SDD)

文法规则:

`switch_stmt` $\rightarrow$ `switch ( expr ) case_list`

`case_list` $\rightarrow$ `case expr : stmt case_list`

$\quad \quad \quad |$ `default : stmt`

$\quad \quad \quad |$ ϵ

`stmt` $\rightarrow$... // 表示其他语句

属性说明:

- 综合属性:

- `case_list.code`: 表示每个 case 分支生成的代码。
- `switch_stmt.code`: 整个 switch 语句生成的代码。

- 继承属性:

- `case_list.next`: 指向 switch 语句结束后的跳转标签。
- `case_list.switch_expr`: 存储 switch 语句的表达式结果。

语义规则:

1. `switch_stmt.code = expr.code || case_list.code`

- 将 switch 表达式的代码和 case_list 的代码结合起来。

2. `case_list[i].code = "if (switch_expr == case_expr) goto Label_i;" || stmt.code || "goto end_switch;"`

- 每个 case 分支生成比较和跳转的代码。

3. `case_list.default.code = stmt.code || "goto`

```
end_switch;"
```

- default 分支直接执行语句。

```
4. case_list.next = newLabel()
```

- next 标记生成用于跳转。

语法制导翻译方案 (SDT)

增强文法规则 (带语义动作):

```
switch_stmt → switch ( expr ) { case_list }
```

```
{
```

```
    switch_stmt.code = expr.code || case_list.code;
```

```
}
```

```
case_list → case expr : stmt case_list
```

```
{
```

```
    case_list1.code = "if ( " switch_expr " == " expr.lexval  
") goto " newLabel() " ;"
```

```
    || stmt.code
```

```
    || "goto " case_list.next;
```

```
    case_list.code = case_list1.code || case_list2.code;
```

```
}
```

```
case_list → default : stmt
```

```
{
```

```
    case_list.code = stmt.code || "goto " case_list.next;
}
```

```
case_list → ε
{
    case_list.code = "";
}
```

1. `switch_stmt → switch ( expr ) { case_list }`

```
switch_stmt → switch ( expr ) { case_list }
{
    switch_stmt.code = expr.code || case_list.code;
}
```

- 解释:
 - `switch_stmt` 是整个 `switch` 语句的起始规则。
 - `expr.code` 表示计算 `switch` 表达式的代码，例如将表达式的值存储在某个寄存器中（如 `R1`）。
 - `case_list.code` 包含所有 `case` 和 `default` 分支的代码。
 - 语义动作：将 `expr.code` 与 `case_list.code` 连接起来，生成完整的 `switch_stmt.code`。
- 作用:
 - 将 `switch` 表达式与各个分支关联起来，为后续生成代码提供基础。
 - 例如，`switch (x)` 的代码可以生成为：

```
text
复制代码
load x, R1
```

2. `case_list → case expr : stmt case_list`

```

case_list → case expr : stmt case_list
{
    case_list1.code = "if ( " switch_expr " == " expr.lexval ") goto "
newLabel() ";"
    || stmt.code
    || "goto " case_list.next;
    case_list.code = case_list1.code || case_list2.code;
}

```

- **解释:**

- case_list 由多个 case 分支组成。
- 每个 case 语句的逻辑包括三个部分:
 1. **条件跳转:** 检查 switch 表达式的值是否等于当前 case 的值, 跳转到该分支对应的标签。
 2. **分支语句:** 执行该分支的 stmt 代码。
 3. **跳转到结尾:** 分支语句执行完后, 跳转到 switch 语句的结束标签, 避免执行后续分支。
- **语义动作:**
 1. case_list1.code:
 - if ("switch_expr == expr.lexval") goto Label;; 生成比较和跳转语句, Label 是为该分支生成的唯一标识符。
 - stmt.code: 生成当前分支的实际语句代码。
 - goto case_list.next: 跳转到 switch 语句的结束标签。
 2. case_list.code: 将当前 case_list1.code 与剩余的 case_list2.code 拼接。

- **作用:**

- 对每个 case 分支生成独立的代码块。
- 例如, case 1: stmt1; 的代码可能生成为:

```

text
复制代码
if (R1 == 1) goto L1;
L1: stmt1;
goto end_switch;

```

3. case_list → default : stmt

```

case_list → default : stmt
{
    case_list.code = stmt.code || "goto " case_list.next;
}

```

- 解释：
 - default 分支的逻辑不需要条件跳转，直接执行 stmt 语句。
 - 执行完 stmt 后跳转到 switch 语句的结束标签。
- 语义动作：
 - stmt.code: 生成 default 分支的语句代码。
 - "goto case_list.next": 直接跳转到结束标签，避免执行后续分支。
- 作用：
 - 提供默认行为，处理所有未被 case 匹配的值。
 - 例如，default: stmt3; 的代码可能生成为：

```
text  
复制代码  
stmt3;  
goto end_switch;
```

4. case_list $\rightarrow \epsilon$

```
case_list  $\rightarrow \epsilon$   
{  
    case_list.code = "";  
}
```

- 解释：
 - 空的 case_list 表示没有定义任何分支，代码为空。
- 语义动作：
 - case_list.code 设为空字符串。
- 作用：
 - 允许文法支持没有 case 或 default 的 switch 语句。

示例解析与生成

输入代码：

```
switch (x) {  
  
    case 1: stmt1;  
  
    case 2: stmt2;
```

```
        default: stmt3;  
    }  
}
```

生成的语法树及代码:

1. 表达式 x 生成代码:

```
load x, R1
```

2. case 1 的代码:

```
if (R1 == 1) goto L1;
```

```
stmt1;
```

```
goto end_switch;
```

3. case 2 的代码:

```
if (R1 == 2) goto L2;
```

```
stmt2;
```

```
goto end_switch;
```

4. default 分支代码:

```
stmt3;
```

```
goto end_switch;
```

5. 结合所有分支:

```
L1: stmt1;
```

```
    goto end_switch;
```

```
L2: stmt2;
```

```
    goto end_switch;
```

```
default: stmt3;
```

```
end_switch;
```